DOSSIER TÉCNICO V2.0

EXPANSIÓN MULTI-API Y DESPLIEGUE NATIVO (ANDROID)

Módulo: Desarrollo de Interfaces (0488) | Ciclo: CFGS Desarrollo de Aplicaciones Multiplataforma (DAM)

Alineación Normativa (Comunidad de Madrid):

- **RA 3:** Crea componentes visuales valorando y empleando herramientas específicas.
  - **CE 3.b:** Se han creado controladores de eventos (gestión asíncrona de peticiones HTTP).
  - **CE 3.d:** Se han enlazado los componentes a orígenes de datos.
- **RA 7:** Prepara y empaqueta aplicaciones para su distribución y despliegue justificando el procedimiento.
  - **CE 7.a:** Se han identificado los elementos necesarios para el empaquetado y la distribución en plataformas móviles.

1. BASE TEÓRICA TÉCNICA

1.1. Escalabilidad del Frontend y Consumo Multi-API

En aplicaciones corporativas, la interfaz rara vez se alimenta de un único origen de datos. Escalar el proyecto para consumir un segundo proveedor REST exige mantener el principio de responsabilidad única (Single Responsibility Principle). Separaremos estrictamente la capa de red (Servicios) de la capa de presentación (Componentes Standalone).

Nota técnica para el aula: El servicio *JSONPlaceholder* es un estándar para pruebas, pero no dispone de un endpoint /products nativo (ofrece /users, /posts, /photos). Para cumplir con el requerimiento de renderizar "productos", utilizaremos la API pública *Fake Store API* (https://fakestoreapi.com/products), que mantiene la misma arquitectura REST y formato JSON, ideal para este escenario.

1.2. De la Web al Dispositivo Nativo (Capacitor)

Ionic utiliza **Capacitor** como motor de ejecución cruzada (Cross-Platform Runtime). Capacitor no compila el código web a lenguajes nativos (como haría Kotlin o Swift), sino que empaqueta la aplicación web compilada dentro de un contenedor nativo (WebView) y expone puentes (Bridges) de JavaScript a las APIs nativas del dispositivo. Esto permite compilar el mismo código fuente web en un .apk o .aab directamente desde Android Studio.

2. ACTIVIDAD GUIADA

PARTE 1: Nueva Página de Productos (Fake Store API)

Paso 1: Modelado de Datos

Definimos la estructura estricta del JSON entrante.
src/app/models/store-product.model.ts

```
TypeScript
export interface StoreProduct {
  id: number;
  title: string;
  price: number;
  description: string;
  category: string;
  image: string;
  rating: {
    rate: number;
    count: number;
  };
}
```

Paso 2: Creación del Servicio Generamos el servicio asíncrono para gestionar las peticiones HTTP. Implementamos la inyección moderna con inject() y retornamos un Observable.
src/app/services/store.service.ts

```
TypeScript
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { StoreProduct } from '../models/store-product.model';

@Injectable({
  providedIn: 'root'
})
export class StoreService {
  // Utilizamos inyección de dependencias moderna en Angular
  private http = inject(HttpClient);
  private apiUrl = 'https://fakestoreapi.com/products';

  getStoreProducts(): Observable<StoreProduct[]> {
    return this.http.get<StoreProduct[]>(this.apiUrl);
  }
}
```

Paso 3: Creación del Componente Standalone Generamos la página que declara exactamente lo que utiliza en sus imports. src/app/pages/catalogo/catalogo.page.ts

TypeScript

```
import { Component, OnInit, inject } from '@angular/core';
import { CurrencyPipe } from '@angular/common';
import {
  IonHeader, IonToolbar, IonTitle, IonContent, IonButtons, IonBackButton,
  IonSpinner, IonCard, IonCardHeader, IonCardTitle, IonCardContent
} from '@ionic/angular/standalone';
import { StoreProduct } from '../../models/store-product.model';
import { StoreService } from '../../services/store.service';

@Component({
  selector: 'app-catalogo',
  templateUrl: './catalogo.page.html',
  styleUrls: ['./catalogo.page.scss'],
  standalone: true,
  imports: [
    CurrencyPipe, IonHeader, IonToolbar, IonTitle, IonContent, IonButtons,
    IonBackButton, IonSpinner, IonCard, IonCardHeader, IonCardTitle, IonCardContent
  ]
})
export class CatalogoPage implements OnInit {
  private storeService = inject(StoreService);

  products: StoreProduct[] = [];
  loading = true;
  error = '';

  ngOnInit() {
    this.fetchProducts();
  }

  fetchProducts() {
    this.storeService.getStoreProducts().subscribe({
      next: (data) => {
        this.products = data;
        this.loading = false;
      },
      error: (err) => {
```

```
        console.error(err);
        this.error = 'Error crítico al conectar con la API';
        this.loading = false;
      }
    });
  }
}
```

Paso 4: Vista Declarativa con Control de Flujo Moderno Utilizamos la sintaxis moderna @if y @for de Angular para optimizar el renderizado sin depender de CommonModule para el control de flujo. src/app/pages/catalogo/catalogo.page.html

HTML

```
<ion-header>
  <ion-toolbar color="tertiary">
    <ion-buttons slot="start">
      <ion-back-button defaultHref="/inicio"></ion-back-button>
    </ion-buttons>
    <ion-title>Catálogo FakeStore</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
  @if (loading) {
    <div class="loading-container" style="text-align: center; padding: 2rem;">
      <ion-spinner name="crescent"></ion-spinner>
      <p>Conectando con FakeStore API...</p>
    </div>
  }

  @if (error) {
    <ion-card color="danger">
      <ion-card-header>
        <ion-card-title>Fallo de Red</ion-card-title>
      </ion-card-header>
      <ion-card-content>{{ error }}</ion-card-content>
    </ion-card>
  }

  @if (!loading && !error) {
    <div class="grid-container" style="display: grid; grid-template-columns: repeat(auto-fill,
minmax(250px, 1fr)); gap: 1rem;">
```

```
      @for (product of products; track product.id) {
        <ion-card>
          <img [src]="product.image" [alt]="product.title" style="height: 200px; object-fit: contain; width:
100%; padding: 10px;" />
          <ion-card-header>
            <ion-card-title style="font-size: 1.1rem;">{{ product.title }}</ion-card-title>
          </ion-card-header>
          <ion-card-content>
            <p style="font-weight: bold; font-size: 1.2rem; color: var(--ion-color-primary);">
              {{ product.price | currency:'EUR' }}
            </p>
            <p>⭐ {{ product.rating.rate }} ({{ product.rating.count }} reviews)</p>
          </ion-card-content>
        </ion-card>
      }
    </div>
  }
</ion-content>
```

Paso 5: Actualización del Enrutador Añadimos la nueva ruta utilizando la carga diferida (loadComponent). src/app/app.routes.ts

```
TypeScript
{
  path: 'catalogo',
  loadComponent: () => import('./pages/catalogo/catalogo.page').then(m => m.CatalogoPage)
}
```

(Se debe añadir también un botón en inicio.page.html con routerLink="/catalogo" para permitir la navegación).

PARTE 2: Despliegue en Emulador con Android Studio

Requisito previo: Android Studio debe estar instalado con al menos un SDK actualizado (API 33+) y un Android Virtual Device (AVD) configurado.

Paso 1: Construcción de la versión de Producción Web

Antes de generar el código nativo, debemos compilar el código TypeScript y HTML de Angular.

```
Bash
ionic build
```

Este comando compilará los assets y los depositará en el directorio www/.

Paso 2: Inicialización e Integración de Capacitor

Si el proyecto no tiene Capacitor habilitado, inicialízalo e instala las dependencias de Android.

```
Bash
npm install @capacitor/android
npx cap add android
```

Esto creará un directorio /android en la raíz del proyecto. Esta carpeta contiene un proyecto nativo de Android basado en Gradle.

Paso 3: Sincronización del Código

Copiamos los archivos compilados de la carpeta www/ al proyecto nativo de Android. **(Este paso debe repetirse cada vez que se modifique el código en Ionic/Angular).**

```
Bash
npx cap sync android
```

Paso 4: Ejecución en Android Studio

Abrimos el IDE directamente desde la línea de comandos de Ionic.

```
Bash
npx cap open android
```

Instrucciones para el usuario dentro de Android Studio:

1. Espera unos minutos a que **Gradle** termine de indexar y construir el proyecto (verás una barra de progreso en la parte inferior derecha).
2. En la barra de herramientas superior, selecciona el emulador deseado (ej. *Pixel 7 API 34*).

3. Haz clic en el botón verde de **Run (Play)** (Atajo: Shift + F10).
4. El emulador se iniciará y la aplicación cargará el WebView mostrando la aplicación Ionic exactamente igual que en el navegador, pero ejecutándose de forma nativa. Podrás interactuar con los listados y verificar que las llamadas HTTP funcionan a través de la red del dispositivo virtual.

3. EJERCICIO PROPUESTO Y RÚBRICA

Práctica de Consolidación:

Desarrollar una nueva funcionalidad que permita filtrar los productos de la FakeStore por categoría.

1. Consumir el endpoint https://fakestoreapi.com/products/categories para obtener un array de categorías.
2. Renderizar botones en la cabecera (usando <ion-segment> o un <ion-select>).
3. Al seleccionar una categoría, filtrar el array localmente utilizando TypeScript (.filter()) o realizando una nueva petición al endpoint https://fakestoreapi.com/products/category/{categoria}.
4. Desplegar los cambios generados y comprobar el funcionamiento en el emulador de Android Studio.

Rúbrica de Evaluación (Basada en CE):

Criterio	Excelente (10-9)	Adecuado (8-6)	Insuficiente (<5)
CE 3.d. Enlace de Datos	Define correctamente las interfaces estrictas e implementa inject(HttpClient) con Observables puros, manejando el ciclo de vida sin memory leaks.	Implementa el consumo pero utiliza tipos any o realiza malas prácticas en la suscripción.	No logra obtener datos dinámicos de la API en la vista.
CE 1.e. Arquitectura Standalone	Mantiene el 100% de los componentes como Standalone, declarando imports granulares y utilizando la sintaxis @if y @for.	Utiliza componentes Standalone pero mantiene sintaxis heredada (*ngIf, *ngFor).	Crea acoplamiento erróneo o requiere módulos clásicos para funcionar.
CE 7.a. Despliegue en Emulador	Sincroniza correctamente con Capacitor, el código compila en Gradle sin errores y la app se ejecuta de forma fluida en el AVD.	Logra abrir Android Studio pero requiere intervención para resolver dependencias de Gradle.	El proyecto no compila en Android o el emulador muestra pantalla en blanco (errores de CORS/WebView).